WebAssembly for browser-based scientific computing: neural simulation and Xiangqi (chinese chess)

Martin D. Pham

University of Toronto, Toronto, ON, Canada
martindopham@gmail.com

Abstract. We present two applications of WebAssembly for browser-based scientific computing. We first present a comparison of the Brian library extension Brian2WASM to a Rust-based spiking neural network implementation compiled to WebAssembly. We then present a WebAssembly module for hyperdimensional computing on the Xiangqi (Chinese chess) board game. Hyperdimensional computing, sometimes called vector symbolic architectures, is a framework for representing data as algebraic operations in a high dimensional vector spaces. We demonstrate preliminary results for board state encoding from Forsyth–Edwards notation and discuss next steps for implementing a vector symbolic game engine/artificial player. Together, these two examples demonstrate how WebAssembly can be deployed on the browser for client-side offloading of scientific computing.

Keywords: WebAssembly · Spiking Neural Network · Hyperdimensional Computing.

1 Introduction and motivation

Browser-based computing is the use of client-side resources, such as the CPU and memory, to perform computations within a web browser. This approach leverages the capabilities of modern web technologies to enable high-performance computing tasks without the need for server-side processing. Recent work has used browser-based tools for drug discovery [37] and for visualizing large datasets [36].

WebAssembly (Wasm) is a binary instruction format that allows code written in high-level languages to be compiled and run in web browsers at near-native speed [13]. It provides a safe and efficient way to execute code on the client side, making it an ideal candidate for scientific computing tasks [26]. Wasm has been used in high-performance computing (HPC) applications [2] and code-generation [10] for big data processing [30].

The remainder of the paper provides background on WebAssembly and the application domains, presents the SNN and HDC methods, reports the results of both experiments, and concludes with directions for future work.

1.1 WebAssembly

WebAssembly is designed to be a portable compilation target for programming languages, enabling high-performance applications on the web [11]. It is supported by all major browsers and provides a secure execution environment through a sandboxed model [42]. Wasm runtimes are responsible for executing WebAssembly code and providing the necessary abstractions for interacting with the host environment. Recent surveys by [44] and [33] provide a comprehensive state of the art overview of WebAssembly runtimes, including their architecture, performance characteristics, and use cases. The work of [24] further surveys the security aspects of WebAssembly, highlighting potential vulnerabilities and mitigation strategies. When compiled from both C++ and Java, Wasm has shown promise in improving performance and resource utilization on the Raspberry Pi [38]. Additionally, Rust [18] may be compiled to Wasm, making it an appealing choice for high-performance web applications. Such results suggest Wasm may be used for similar gains in browser-based computing, although there is still a need for more research in this area to fully understand the trade-offs and best practices for using WebAssembly in scientific computing applications.

1.2 Spiking Neural Networks

Spiking neural networks are a class of artificial neural networks that explicitly model temporal dynamics of neural activity, making them more biologically plausible than traditional artificial neural networks [41]. They are particularly well-suited for tasks involving time-varying signals and have been applied in various domains, including robotics, sensory processing, and neuromorphic computing. Neural activity is modeled as discrete events, or spikes, which occur at specific points in time and are driven by individual neuronal models [9]. This event-driven approach allows for more efficient processing of temporal information compared to traditional methods. However, training SNNs can be challenging due to the non-differentiable nature of spike events. Frameworks such as Brian use a code-generation strategy to initially represent the network and simulation definition as an abstract syntax tree before generating numerical solvers that may be used at runtime [31].

The temporally sparse representation of activity allows for low resource utilization, making SNNs a promising approach for online [22] and edge computing applications [43] [40] when combined with appropriate hardware. Spike-based models are of particular interest for their applications in neuromorphic computing on non-Von Neumann machines [17], however there is still many open problems regarding SNN dynamics and compilations. While many SNN simulators exist [39], there is still a need for more efficient and scalable solutions to fully leverage the potential of SNNs in practical applications and on traditional hardware environments such as client-side browsers.

1.3 Hyperdimensional Computing

Hyperdimensional computing, also known interchangeably as vector symbolic architectures (VSA), is a computational framework that represents data as high-dimensional vectors, typically with thousands of dimensions [14]. These hyperdimensional vectors are usually drawn from random projections, and with appropriate binding, bundling and other vector symbolic operations, they can be algebraically manipulated to represent complex data structures and relationships [15]. The work of [35] provides some theoretical framing of HDCs, and the work of [28] provides a foundation for a category theoretic understanding of VSAs. A comprehensive pair of surveys was done by [20] and [19] to summarize the state of the art in models and applications of HDCs, with a useful comparison across different VSAs done by [27].

Although libraries such as TorchHD [12] provide a platform to implement different VSAs, it remains a challenge to create efficient implementations that can be deployed in web browsers for client-side computing. An important approach is the one taken by Nengo [1], which uses the Neural Engineering Framework (NEF) [8] to implement a VSA called the Semantic Pointer Architecture (SPA) [7], a variation of Holographic Reduced Representations (HRR) [25] which uses SNNs. The Nengo software provides a frontend application based on a Python web server, which interprets the model and streams results back as they are computed.

1.4 Xiangqi (Chinese Chess)

Computer Chinese chess [5] is the computational study and implementation of the board game Xiangqi [16]. Strategies include pruning [34] and search-based algorithms [32], coevolutionary approaches [23], and more recently deep [45] and reinforcement learning [21]. Computer vision and robotic applications have have been successfully deployed using convolutional neural networks [4]. A recent perspective report by [3] provides an overview of the advancements and challenges in the field.

2 Methods

2.1 Brian2 vs Rust-based Simulators

Brian2Wasm [29] is a library extension for the Brian simulator which targets WebAssembly for compilation. Brian2 provides equation-based model specification with automatic code genera-

tion, including compiled backends; its standalone C++ mode can use threaded execution with suitable toolchains, while Brian2Wasm deploys generated kernels in the browser. The neuron model simulated is the Ornstein-Uhlenbeck process as seen in Figure 1, where y is the state variable, τ is the time constant, σ is the noise intensity, and ξ is a Gaussian white noise term. We simulate a neural population of 5000 neurons for 10 seconds of simulation time. We evaluate wall clock time for Brian-based and Rust-based implementations of this process across four configurations in Figure 3: Brian native, Brian2Wasm in browser, Rust native, and Rust+Wasm in browser. Native solvers use compiled/optimized builds, while browser solvers use WebAssembly. WebAssembly supports SIMD and thread-level parallelism via shared memory and atomics, but due to browser deployment constraints (cross-origin isolation), the reported browser runs use a single-threaded configuration.

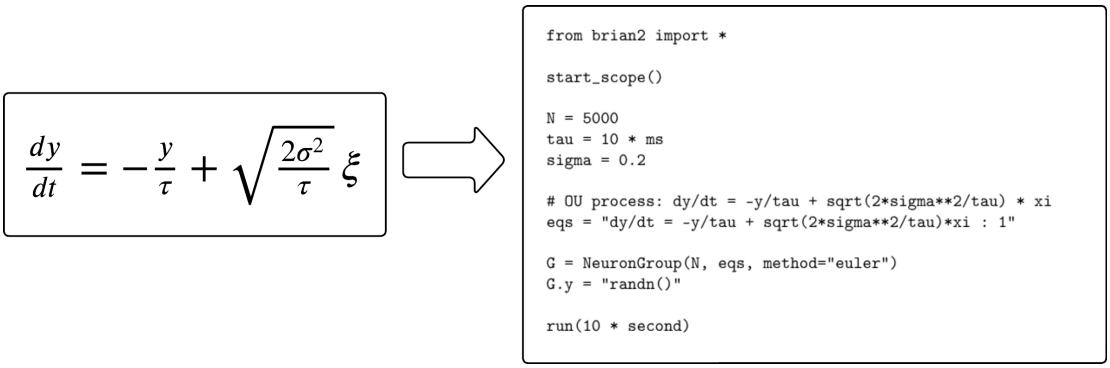

Fig. 1. Brian2 code snippet for the Ornstein-Uhlenbeck process, which can be compiled to WebAssembly for browser deployment.

Compared directly, vanilla Brian2 (native backend) prioritizes speed and parallel tuning, while Brian2Wasm prioritizes deployability and reproducibility in the browser. Vanilla Brian2 can use generated C++ and host-side threading to maximize throughput, whereas Brian2Wasm runs inside the browser sandbox, where threading depends on deployment constraints and can add runtime overhead. Relative to Brian, a Rust implementation requires explicit numerical integration logic, random-noise handling, memory layout choices, and separate native and Wasm build/tooling pipelines. This increases implementation effort but gives tighter low-level control over data structures and optimization, whereas Brian offers faster model iteration through high-level equations and automatic solver/code generation. In practice, optimization emphasis shifts by deployment: vanilla Brian favors solver/method selection and generated-backend tuning, Brian2Wasm favors browser-facing constraints (payload size, startup cost, and runtime compatibility), Rust native favors architecture-specific compiler/vectorization and memory-layout tuning, and Rust+Wasm favors Wasm-targeted size/performance tradeoffs (for example SIMD enablement, bindgen boundary minimization, and reduced host-call overhead). We intentionally deferred basic low-level optimizations (manual SIMD kernels, aggressive loop fusion/unrolling, and multi-threaded browser execution) in this comparison to preserve a like-for-like baseline across all four columns; computations are primarily IEEE-754 double precision as provided by Brian/Python and Rust defaults, and explicit vectorized operations were not hand-implemented beyond compiler/runtime auto-vectorization when available.

2.2 Xiangqi Hyperdimensional Encoding

Using a modified version of the Forsyth–Edwards Notation (FEN) [6], we encode the state of a Xiangqi board as a hyperdimensional vector as seen in Figure 2. The FEN string representing the board state is given by:

`rnbakabnr/9/1c5c1/p1p1p1p1p/9/9/P1P1P1P1P/1C5C1/9/RNBAKABNR`

where rows are separated by /, lowercase letters represent red pieces, uppercase letters represent black pieces, and numbers represent empty squares. By using a role-filler construction, we bind each piece to its position on the board and bundle all piece-position pairs into a single hyperdimensional vector representing the entire board state.

Let $\mathcal{T} = \{\text{RED}, \text{BLACK}\}$ be team symbols, $\mathcal{P} = \{K, A, B, N, R, C, P\}$ be piece symbols, and let X and Y be base row and column hypervectors. Let Π denote a fixed permutation operator, so that $\Pi^i(X)$ and $\Pi^j(Y)$ encode coordinate-dependent row/column roles. With binding operator $\otimes$ and bundling operator $\oplus$, each occupied square contributes

$$v_{i,j} = (t_{i,j} \otimes p_{i,j}) \otimes \left(\Pi^i(X) \otimes \Pi^j(Y)\right),$$

and the board representation is

$$V_{\text{board}} = \bigoplus_{(i,j)\in\Omega} v_{i,j},$$

where Ω is the set of occupied coordinates. To query the piece at a location (i, j), we unbind position and decode by similarity over the team-piece dictionary:

$$\tilde{u}_{i,j} = V_{\text{board}} \otimes \left(\Pi^i(X) \otimes \Pi^j(Y)\right)^{-1}, \qquad (\hat{t}, \hat{p}) = \arg\max_{t\in\mathcal{T},\, p\in\mathcal{P}} \text{sim}(\tilde{u}_{i,j},\, t \otimes p).$$

Conversely, to query location from a team-piece token (t, p), we compute

$$\tilde{\ell}_{t,p} = V_{\text{board}} \otimes (t \otimes p)^{-1}$$

and select the coordinate pair (i, j) with maximal similarity to $\left(\Pi^i(X) \otimes \Pi^j(Y)\right)$.

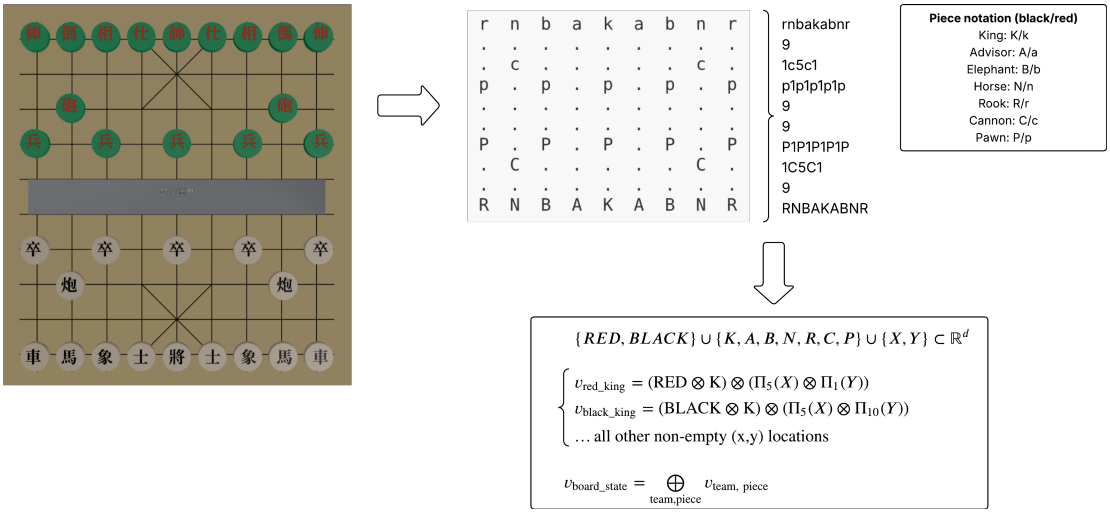

Fig. 2. The board state is first converted into a text string (FEN) and then encoded into a hyperdimensional vector using a location and piece-based role-filler construction.

3 Results and Discussion

3.1 Comparing Neural Solvers

Both browser-based and native implementations were evaluated over 100 runs. Browser-based simulations took longer than native ones, but the Rust-based Wasm implementation remained competitive despite the lack of additional optimization. Figure 3 shows the performance comparison of Brian2WASM and Rust-based SNN implementations for simulating the Ornstein-Uhlenbeck process.

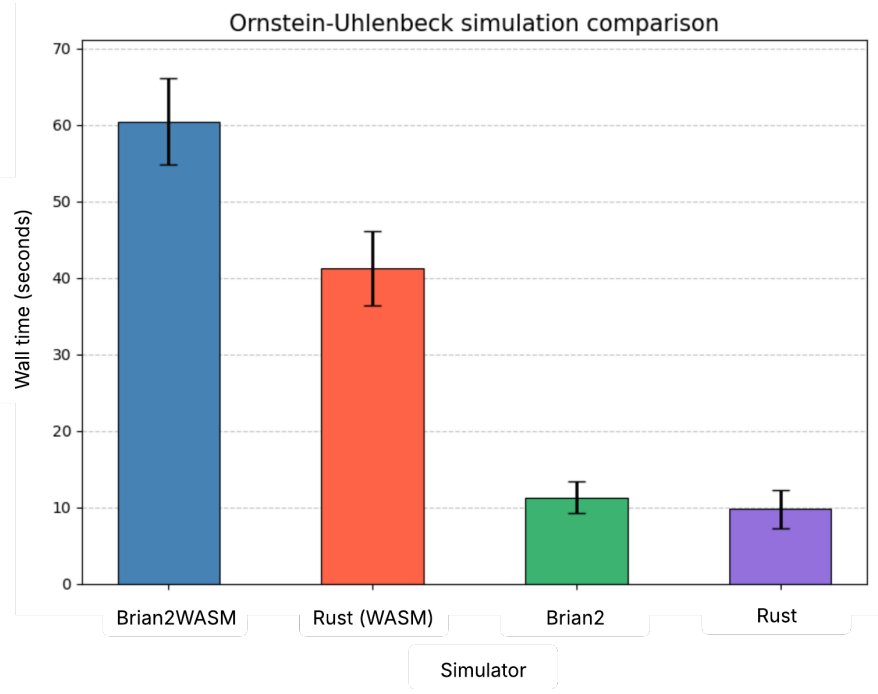

Fig. 3. Performance comparison of Brian2WASM and Rust-based SNN implementations for simulating the Ornstein-Uhlenbeck process over 100 runs. All simulations were run on a System76 Lemur Pro with 56Gb of RAM. Browser based simulations were run in Firefox. In all four columns, the computational core is compiled code (generated Brian kernels or Rust release binaries/Wasm modules), not interpreted numerical loops.

3.2 Xiangqi Querying

The computed distributed hypervector encoding the board state was queried against all 90 possible piece positions on the board. Over 100 runs, the encoding performed poorly, obtaining an average accuracy of only 24%($\pm$7%) in correctly identifying the piece at a given position. Many errors arose from misidentifying empty squares as occupied, likely because the encoding does not include an explicit representation for empty positions. The role-filler construction used for position vectors may also limit recall accuracy relative to alternative position encodings.

4 Conclusion and Future Work

We presented two WebAssembly-based case studies to evaluate Rust as a practical alternative to Brian2WASM for browser deployment of scientific workloads. For the SNN study, browser execution incurred a clear overhead relative to native runs (up to about 6$\times$ in our setting), but the Rust-based Wasm implementation remained functionally correct and competitive with Brian2WASM for client-side use. These results support browser-native deployment as a useful option when reducing server-side setup and scaling burden is important for shared simulation applications. For Xiangqi, the low query accuracy should be interpreted as a limitation of the current HDC encoding design rather than a refutation of the browser/Wasm approach, motivating future work on richer empty-square representations and alternative position-composition schemes.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Bekolay, T., Bergstra, J., Hunsberger, E., DeWolf, T., Stewart, T.C., Rasmussen, D., Choo, X., Voelker, A.R., Eliasmith, C.: Nengo: a python tool for building large-scale functional brain models. Frontiers in neuroinformatics **7**, 48 (2014)
2. Chadha, M., Krueger, N., John, J., Jindal, A., Gerndt, M., Benedict, S.: Exploring the use of webassembly in hpc. In: Proceedings of the 28th ACM SIGPLAN annual symposium on principles and practice of parallel programming. pp. 92–106 (2023)
3. Chen, B.N., Chen, C.H., Hung, C.H., Tseng, W.J.: Computer chinese chess: Past, present, and future. ICGA Journal **47**(2), 68–94 (2025)
4. Chen, P.J., Yang, S.Y., Wang, C.S., Muslikhin, M., Wang, M.S.: Development of a chinese chess robotic system for the elderly using convolutional neural networks. Sustainability **12**(10), 3980 (2020)
5. Chen, S.J.Y.J.C., Yang, T.N., Hsu, S.C.: Computer chinese chess. ICGA journal **27**(1), 3–18 (2004)
6. Edwards, S.J.: Portable game notation specification and implementation guide. Retrieved April **4**, 2011 (1994)
7. Eliasmith, C.: How to build a brain: A neural architecture for biological cognition. OUP USA (2013)
8. Eliasmith, C., Anderson, C.H.: Neural engineering: Computation, representation, and dynamics in neurobiological systems. MIT press (2003)
9. Fortuna, L., Buscarino, A.: Spiking neuron mathematical models: a compact overview. Bioengineering **10**(2), 174 (2023)
10. Groppe, S., Reimer, N.: Code generation for big data processing in the web using webassembly. Open Journal of Cloud Computing (OJCC) **6**(1), 1–15 (2019)
11. Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A., Bastien, J.F.: Bringing the web up to speed with webassembly. In: Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation. pp. 185–200 (2017)
12. Heddes, M., Nunes, I., Vergés, P., Kleyko, D., Abraham, D., Givargis, T., Nicolau, A., Veidenbaum, A.: Torchhd: An open source python library to support research on hyperdimensional computing and vector symbolic architectures. Journal of Machine Learning Research **24**(255), 1–10 (2023)
13. Herrera, D., Chen, H., Lavoie, E., Hendren, L.: Webassembly and javascript challenge: Numerical program performance using modern browser technologies and devices. University of McGill, Montreal: QC, Technical report SABLE-TR-2018-2 (2018)
14. Kanerva, P.: Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. Cognitive computation **1**(2), 139–159 (2009)
15. Kanerva, P.: Hyperdimensional computing: An algebra for computing with vectors. Advances in Semiconductor Technologies: Selected Topics Beyond Conventional CMOS pp. 25–42 (2022)
16. Khan, S.U.: Plays, values, analysis and the complexity of chinese chess. Bulletin of the EATCS **81**, 253–263 (2003)
17. Kim, S., Cho, J., Shin, J., Kim, B., Jung, J.: Automatic generation of spiking neural networks on neuromorphic computing hardware for iot edge computing. Future Generation Computer Systems **174**, 107953 (2026)
18. Klabnik, S., Nichols, C.: The Rust programming language. No Starch Press (2023)
19. Kleyko, D., Rachkovskij, D., Osipov, E., Rahimi, A.: A survey on hyperdimensional computing aka vector symbolic architectures, part ii: Applications, cognitive models, and challenges. ACM Computing Surveys **55**(9), 1–52 (2023)
20. Kleyko, D., Rachkovskij, D.A., Osipov, E., Rahimi, A.: A survey on hyperdimensional computing aka vector symbolic architectures, part i: Models and data transformations. ACM Computing Surveys **55**(6), 1–40 (2022)
21. Li, M., Huang, W.: Research and implementation of chinese chess game algorithm based on reinforcement learning. In: 2020 5th International Conference on Control, Robotics and Cybernetics (CRC). pp. 81–86. IEEE (2020)
22. Lobo, J.L., Del Ser, J., Bifet, A., Kasabov, N.: Spiking neural networks and online learning: An overview and perspectives. Neural Networks **121**, 88–100 (2020)
23. Ong, C.S., Quek, H., Tan, K.C., Tay, A.: Discovering chinese chess strategies through coevolutionary approaches. In: 2007 IEEE Symposium on Computational Intelligence and Games. pp. 360–367. IEEE (2007)
24. Perrone, G., Romano, S.P.: Webassembly and security: A review. Computer Science Review **56**, 100728 (2025)
25. Plate, T.A.: Holographic reduced representations. IEEE Transactions on Neural networks **6**(3), 623–641 (1995)

26. Rusum, G.P.: Webassembly across platforms: Running native apps in the browser, cloud, and edge. International Journal of Emerging Trends in Computer Science and Information Technology **3**(1), 107–115 (2022)
27. Schlegel, K., Neubert, P., Protzel, P.: A comparison of vector symbolic architectures. Artificial Intelligence Review **55**(6), 4523–4555 (2022)
28. Shaw, N.P., Furlong, P.M., Anderson, B., Orchard, J.: Developing a foundation of vector symbolic architectures using category theory. arXiv preprint arXiv:2501.05368 (2025)
29. simulator, B.: brian2wasm (2022), https://github.com/brian-team/brian2wasm, accessed 2026-03-21
30. Sok, C., d'Orazio, L., Tekin, R., Tombroff, D.: Webassembly serverless join: A study of its application. In: Proceedings of the 36th International Conference on Scientific and Statistical Database Management. pp. 1–4 (2024)
31. Stimberg, M., Brette, R., Goodman, D.F.: Brian 2, an intuitive and efficient neural simulator. elife **8**, e47314 (2019)
32. Su, K.L., Chung, C.Y., Zou, J.T., Hsu, T.Y.: A* search algorithm applied to a chinese chess game. Artificial Life and Robotics **16**(2), 132–136 (2011)
33. Ţălu, M.: A comparative study of webassembly runtimes: performance metrics, integration challenges, application domains, and security features. Archives of Advanced Engineering Science pp. 1–13 (2025)
34. Tao, J., Wu, G., Pan, X.: Design and improvement of the pruning algorithm of the chinese chess in the computer games. The Journal of Engineering **2020**(13), 426–428 (2020)
35. Thomas, A., Dasgupta, S., Rosing, T.: A theoretical perspective on hyperdimensional computing. Journal of Artificial Intelligence Research **72**, 215–249 (2021)
36. Usher, W., Pascucci, V.: Interactive visualization of terascale data in the browser: Fact or fiction? In: 2020 IEEE 10th Symposium on Large Data Analysis and Visualization (LDAV). pp. 27–36. IEEE (2020)
37. Wang, A., Durrant, J.D.: Open-source browser-based tools for structure-based computer-aided drug discovery. Molecules **27**(14), 4623 (2022)
38. Wang, B., Li, H., Wang, J.: High-performance computing in browsers: An empirical study evaluating the computational efficiency of js, c++ wasm, and java wasm in matrix operations. In: 2025 6th International Conference on Intelligent Computing and Human-Computer Interaction (ICHCI). pp. 366–373. IEEE (2025)
39. Wu, J., Wang, Y., Li, Z., Lu, L., Li, Q.: A review of computing with spiking neural networks. Computers, Materials & Continua **78**(3) (2024)
40. Xue, J., Xie, L., Chen, F., Wu, L., Tian, Q., Zhou, Y., Ying, R., Liu, P.: Edgemap: an optimized mapping toolchain for spiking neural network in edge computing. Sensors **23**(14), 6548 (2023)
41. Yamazaki, K., Vo-Ho, V.K., Bulsara, D., Le, N.: Spiking neural networks and their applications: A review. Brain sciences **12**(7), 863 (2022)
42. Yan, Y., Tu, T., Zhao, L., Zhou, Y., Wang, W.: Understanding the performance of webassembly applications. In: Proceedings of the 21st ACM Internet Measurement Conference. pp. 533–549 (2021)
43. Zhang, G., Feng, L., Zhou, F., Yang, Z., Zhang, Q., Saleh, A., Donta, P.K., Dehury, C.K.: Spiking neural networks in intelligent edge computing. IEEE Consumer Electronics Magazine **14**(4), 66–75 (2024)
44. Zhang, Y., Liu, M., Wang, H., Ma, Y., Huang, G., Liu, X.: Research on webassembly runtimes: A survey. ACM Transactions on Software Engineering and Methodology **34**(8), 1–47 (2025)
45. Zhou, C.: Xqsv: A structurally variable network to imitate human play in xiangqi. In: 2024 IEEE Conference on Games (CoG). pp. 1–6. IEEE (2024)